

Welcome to Python: A Beginner's Guide

Introduction: What is Python?

Python is an open-source, high-level programming language known for its simplicity and versatility. It is a **general-purpose** language, meaning you can use it for almost any kind of software development – from web applications (e.g. using the Django framework) to scientific computing and data analysis [1]. First released in 1991, Python has become one of the most popular and recommended languages for beginners and experts alike. Its syntax is designed to be highly readable (often resembling plain English), which makes it easy to learn and **beginner-friendly** [2]. Python's popularity is backed by a large, active community and a rich ecosystem of libraries for various domains (web, data science, machine learning, automation, etc.) [3]. Many top companies (like Dropbox, Disney, Spotify, Netflix, and NASA) use Python in their technology stacks [4], and Python consistently ranks among the most-loved and in-demand programming technologies in surveys [5]. In short, Python is a powerful yet accessible language that can open the door to many areas of computing.

Approaches to Using Python: From Simple to Advanced Workflows

When starting out, there are multiple ways to **set up and work with Python**, each with its own pros and cons. Below we examine several approaches, from the simplest (no installation) to more complex development workflows:

1. Online Python Environments (Cloud Notebooks)

One of the easiest ways to start coding in Python is to use an **online environment**. Tools like **Google Colab** or **Jupyter notebooks on cloud platforms** allow you to write and run Python code through your web browser, with no installation on your computer. This approach is beginner-friendly and convenient for quick experiments or learning:

- **Pros:** No need to install anything locally – you can start coding immediately in a browser. Colab, for example, provides free access to computing resources (even GPUs/TPUs) and comes with many libraries pre-installed [6]. It's easy to share notebooks and collaborate, since everything is stored online in your Google Drive. You

also don't have to worry about managing packages or dependencies initially, as the environment is pre-configured [7].

- **Cons:** Requires a stable internet connection to use [8]. You are limited by the cloud environment's restrictions (runtime limits, certain filesystems). Data privacy could be a concern if you're uploading sensitive data to the cloud [9]. Additionally, you have less control over the environment (for example, you might not be able to install arbitrary system tools). Performance can be limited for very large datasets since you depend on remote servers and network speed [10]. In summary, *Google Colab is cloud-based while Jupyter notebooks run on your local machine* – Colab offers instant setup and powerful resources, whereas local notebooks offer more control and offline access[11].

When to use: Online notebooks are great for initial learning, short projects, or when you can't install software on your device. Educators often use them so students can focus on coding rather than environment setup. If you have internet access and want to start right away, this is the simplest route.

2. Installing Python Locally (Standard Installation)

Another approach is to **install Python on your own computer** and run code locally. The official Python distribution can be downloaded from Python's website (<https://www.python.org/downloads>) for Windows, macOS, or Linux. This approach gives you a persistent Python setup on your machine:

- **Pros:** Full control over your environment, and you can work offline. The official installer comes with the **Python interpreter** and the **IDLE** development environment (a simple code editor) included [12]. You can run Python programs from your command line or terminal, and install additional libraries as needed using Python's package manager (pip). Running code locally can be faster for data stored on your computer, and you're not constrained by cloud session limits.
- **Cons:** Requires you to download and **install** software, which might be challenging if you don't have admin rights or experience with installations. You also need to manage updates and package installations yourself. On Windows, a common hurdle is ensuring the *PATH* is set correctly (so that the python command is recognized in the terminal) – fortunately, the installer has an option “Add Python to PATH” to simplify this [13]. Another consideration is that you might encounter version conflicts or need to set up a **virtual environment** to isolate project dependencies as you install more packages (this is more of an issue in advanced use, but worth noting).

When to use: Installing Python locally is recommended if you plan to do substantial work on your own machine, want the ability to save files locally, or if you have offline requirements. It's the next step once you outgrow the simplicity of online notebooks and need more control or want to build larger projects.

3. Using a Python Distribution (Anaconda for Data Science)

For learners interested in data science or scientific computing, an **all-in-one distribution** like **Anaconda** is a popular choice. Anaconda bundles Python with a collection of commonly used libraries (NumPy, Pandas, matplotlib, etc.) and includes tools like **Jupyter Notebook** and **JupyterLab** by default.

- **Pros:** One installation gives you a ready-to-use environment for data analysis and machine learning. Many packages come pre-installed, so you avoid dealing with dependency issues initially. It also provides a user-friendly launcher (Anaconda Navigator) to open notebooks or other tools without command-line usage. This can simplify setup for beginners focusing on data tasks.
- **Cons:** The download size is large (hundreds of MBs) because of all the bundled packages. It may install some packages you don't need, using significant disk space. Anaconda manages packages with its own tool (conda), which means you might need to learn a slightly different workflow than using pip. Mixing conda and pip can sometimes lead to environment conflicts, so it requires a bit of care. Additionally, because it's a separate distribution, it might not always have the very latest Python version or package releases (though it's usually up-to-date).

When to use: If your primary goal is data science or you want a hassle-free way to get a scientific Python stack, Anaconda is a good choice. It's often used in academic settings for its convenience. However, for general programming or if you prefer a lean setup, the standard Python installation plus manual package installs might be preferable.

4. Integrated Development Environments (IDEs) and Code Editors

As you progress, you may want to use a more powerful code editor or **IDE** (Integrated Development Environment) for writing Python code. Tools like **Visual Studio Code (VS Code)**, **PyCharm**, or **Spyder** offer rich features like syntax highlighting, auto-completion, debugging, and project management.

- **Pros:** An IDE can greatly enhance productivity for larger projects. Features like IntelliSense (auto-complete suggestions), real-time error highlighting (linting), and integrated debugging help catch mistakes and understand code faster [14]. Many IDEs

also integrate with version control (Git) and have plugins for Python that make tasks like testing or refactoring easier [15]. VS Code, for instance, is lightweight yet powerful – with the official Python extension, it becomes a versatile Python IDE.

- **Cons:** There is a learning curve to using an IDE effectively. Beginners might find the interface and options overwhelming at first. You also have to install and configure the IDE in addition to Python, which is an extra step. Some IDEs (like PyCharm) are heavier on system resources, which might be an issue on older or low-spec computers. It's easy to stick with simpler editors at the very beginning and then transition to an IDE when projects become more complex.

When to use: If you start working on multi-file projects, need to debug code, or just prefer a more feature-rich environment, an IDE is appropriate. Many beginners start with IDLE or simple text editors and then move to VS Code or PyCharm as they become more comfortable. Visual Studio Code in particular is a friendly stepping stone – you can use it just as a text editor initially, and gradually make use of its IDE features as needed.

5. Advanced Workflows: Virtual Environments and Cloud Deployment

For completeness, it's worth mentioning what more advanced Python workflows look like (even if it's beyond the initial learning phase):

- **Virtual Environments:** As you install more packages, you'll encounter situations where different projects require different package versions. **Virtual environments** are isolated spaces for project-specific dependencies. Tools like `python -m venv` (built-in) or `conda env` or `pyenv` help manage this. For example, you might create a virtual environment for a web app project so that its dependencies don't conflict with those of a data science project. Using virtual environments is considered a best practice in Python development[16][17], but if you're just starting and using one environment for everything, you may not need this immediately.
- **Cloud Deployment:** Eventually, you may want to turn your Python code into an application that others can use (for example, a web service or a data dashboard). Python supports this through web frameworks like **Flask** or **Django**, and you can deploy apps to cloud platforms (AWS, Heroku, etc.) or as simple cloud functions. This is an advanced step, but it's the direction many projects go. Thanks to Python's flexibility, the same code you prototype in a notebook can often be integrated into a web application or cloud workflow. (For instance, Python is commonly used in web development and can be the foundation of full software products[18].) Mastering the basics will prepare you to explore these deployment tools when you're ready.

In summary, **beginners should start with the approach that gets them coding as easily as possible** – an online notebook or a basic local install – and gradually adopt more sophisticated tools as needed. Python’s strength is that it scales with you: you can start with a single notebook cell and, down the road, end up building a complex cloud-deployed application using the same core language.

Installing Python

On macOS (Apple M Silicon)

If you are using a Mac with an Apple Silicon chip (M1, M2, etc.), installing Python is straightforward. Newer versions of macOS (since macOS Catalina) do *not* come with a usable Python 3 installation by default [19], so you will need to install Python 3 yourself. There are a couple of methods:

Method 1: Install via Homebrew (Recommended for Developers). If you’re comfortable with the Terminal, you can use Homebrew, a package manager for macOS, to get Python. First, install Homebrew (if not already installed) by running the official install script in Terminal [20]. Then install Python by typing:

```
brew install python3
```

This will fetch the latest Python 3 version. Homebrew makes it easy to update Python later (brew upgrade python) and manages the installation for you [21][22].

Method 2: Download the Official Python Installer. The Python Software Foundation provides macOS installers on the [official Python website](#). Visit the downloads page and click the Python 3 download button for macOS [23]. Make sure to choose the **universal installer** package, which supports both Intel and Apple Silicon Macs (modern Python installers include native support for Apple’s M1/M2 chips) [24]. Download the .pkg file and run it, then follow the prompts through the installation wizard (the default settings are usually fine) [25]. You may be asked for your Mac login password to approve the install.

Apple Silicon Note: The universal installer will install an optimized version of Python for your M1/M2 Mac. In the past, there were workarounds using Rosetta or specific builds, but nowadays the official Python 3.11+ installer works natively on Apple Silicon Macs [24], so you get full performance out of Python.

After installation completes, you can verify it was successful. One way is to open the **Terminal** (find it in Applications > Utilities) and type `python3 --version` (or `python3` to enter the interactive shell) [26]. Another way is to use **IDLE**, the Integrated Development Environment that comes

with Python. Find IDLE in your Applications (it might be in a folder called “Python 3.x”). Launch IDLE, and it will open a window with the Python prompt `>>>`. You can try typing a simple command like `print("Hello, Python!")` and see the output. If everything is installed correctly, IDLE will show the Python shell waiting for your commands, as illustrated below:

After installing Python on macOS, you can launch IDLE. This is the Python 3 shell on a Mac, indicating a successful installation. [12]

(If the version number in IDLE or the Terminal is displayed (e.g. “Python 3.11.4 ...”), you have Python set up on your Mac!)

Tip: By default, the `python` command on macOS might still refer to Python 2 (on some older systems). It’s safer to use `python3` to run Python 3. Also, consider installing a code editor or IDE (like VS Code, mentioned earlier) for a better coding experience once Python is set up.

Method 3: Install via Anaconda Distribution.

Anaconda is a popular all-in-one Python distribution designed for **data science and machine learning**. It comes with Python plus hundreds of commonly used libraries and tools already installed, including **Jupyter Notebook** and **JupyterLab**.

- Download the **Anaconda installer for macOS (Apple Silicon)** from the Anaconda website.
- Run the installer and follow the instructions (defaults are fine).
- After installation, you can open **Anaconda Navigator**, a graphical interface for launching Jupyter, Spyder, or managing environments.

Pros:

- Beginner-friendly: Python, Jupyter, and common libraries are included out of the box.
- Optimized for **data science and research workflows**, making it a good fit for this course.
- Works natively on M1/M2 chips with the latest builds.
- Let you manage multiple isolated environments easily with `conda`.

Cons:

- Large download size (several hundred MB), taking more disk space by default.

Installing Python on Windows

Installing Python on Windows is also easy, with two main options available. You can either use the **Microsoft Store app** or install Python from the **official website**. We’ll cover both:

Option 1: Install from the Microsoft Store (Windows 10/11). Microsoft provides Python via the Windows Store, which is a quick way to get Python 3 set up without dealing with PATH settings or admin permissions. Simply open the Microsoft Store, search for **Python 3**. (look for the one published by the Python Software Foundation), and click **Get/Install**[27]. This will install Python for your user account. The Store installation automatically configures your PATH, so you can immediately use the python command in PowerShell or Command Prompt[28]. It also will automatically update Python when new versions are released. This method is recommended for beginners by Microsoft's documentation because it's simple and doesn't require extra configuration[29]. After installation, you can verify by opening a new PowerShell (or Command Prompt) and typing `python --version` to see the installed version[30]. You should also find **IDLE** in your Start menu (e.g. "IDLE (Python 3.x)") which you can launch to bring up the Python shell.

Option 2: Install using the Official Installer from Python.org. Go to python.org/downloads and download the latest Windows installer (choose the 64-bit installer for modern PCs)[31]. Run the installer .exe file. **Important:** On the installer's first screen, check the box that says **"Add Python to PATH"** (and optionally "Install launcher for all users") before clicking "Install Now"[13]. This ensures that the Python executable is added to your system PATH, which means you can use the python command from any terminal window. The installer will then copy Python onto your system. After a few minutes, you should see a success message[32].

During the Python installation on Windows, make sure to check the option to Add Python to PATH (as shown above). This will let you run python from the command line without extra steps.[13]

Once installed, verify it by opening the **Command Prompt** (cmd.exe) and running `python --version`, which should display the Python 3 version. You can also launch the **IDLE** app (search for "IDLE" in Start) – if it opens and shows a Python shell window, the installation was successful[33]. From the command line, the `python` (or `python3`) command will start the interactive interpreter, where you'll see the `>>>` prompt. Type `exit()` or press Ctrl-Z then Enter to quit the interactive mode.

Option 3: (Alternative) Anaconda Distribution. If you prefer using Anaconda on Windows, download the Anaconda installer from anaconda.com (it's a large .exe file). Running it will install Python and a suite of tools. Anaconda has its own Start menu entry (Anaconda Prompt, Jupyter Notebook, etc.). This is an alternative to the above, mainly for those focusing on scientific computing.

After installing via any method, you are ready to write and run Python code on Windows. If you plan to use a code editor like VS Code, be sure to install it and the Python extension as

mentioned earlier. Visual Studio Code can detect your Python installation automatically, especially if Python is in the PATH.

Note: For web development on Windows, you might encounter advice to use the Windows Subsystem for Linux (WSL) to run Python in a Linux environment[34]. That's an advanced setup for specific needs (e.g. if deploying on Linux servers), and not necessary for beginners. The native Windows Python will work for learning purposes and most tasks.

Working with Python: Running Scripts and Notebooks

With Python installed (or an online environment ready), you can now start **running Python code**. There are a few common ways to use Python interactively or run programs:

- **Interactive mode (REPL):** If you type `python` in a terminal/command prompt, you enter the interactive mode, also known as the REPL (Read-Eval-Print Loop) or simply the Python shell. You'll see a `>>>` prompt. You can type Python statements one by one and see results immediately. This is useful for quick experiments or calculations. Both IDLE and the terminal `python` command provide this interactive interpreter[35]. For example, entering `2 + 2` will instantly output `4`. To exit the interactive mode, use `exit()`.
- **Running scripts:** You can write Python code in a text file with a `.py` extension (this is called a *script*) and run it as a whole. For instance, you might create a file `hello.py` containing `print("Hello")`. Running this script can be done by executing `python hello.py` in a terminal. If using an IDE or code editor, there are typically run commands (for example, in VS Code you can press the Run button or F5 to execute the script). This mode is better for saving and re-running programs, and it's how larger applications are developed.
- **Jupyter Notebooks:** A Jupyter Notebook is an interactive web-based interface that lets you mix code, text, and visualizations. It's extremely popular in education and data science. You can start a local Jupyter Notebook server by installing the `notebook` package (`pip install notebook`) and running `jupyter notebook`. This will open a browser interface where you can create notebooks. In a notebook, you write code in cells and execute them to see output below the cell. Notebooks are great for step-by-step exploration and keeping notes alongside code. If you installed Anaconda, Jupyter is already included (you can launch it from the Anaconda Navigator or the Start menu on Windows). If you prefer not to install anything, remember you can use **Google Colab**, which is essentially a hosted Jupyter Notebook provided by Google, accessible at colab.research.google.com.

- **Installing Jupyter Notebook (for Method 1 & 2 users)**

If you installed Python via **Homebrew** or the **official .pkg installer**, Jupyter Notebook is **not included by default**. You need to install it separately:

1. Open **Terminal**.
2. Upgrade pip (optional but recommended):

```
python3 -m pip install --upgrade pip
```

3. Install the Jupyter Notebook package:

```
python3 -m pip install notebook
```

4. To launch Jupyter, navigate to your project folder in Terminal and type:

```
jupyter notebook
```

This will open a browser window at <http://localhost:8888>, where you can create and run notebooks.

Note: If you used **Anaconda (Method 3)**, Jupyter is already installed — you can launch it directly from Anaconda Navigator or by typing `jupyter notebook` in Terminal.

When to use notebooks vs scripts: For learning and experimentation, notebooks are fantastic because you can run pieces of code in isolation and see results (graphs, data tables, etc.) immediately. Our guide will next have you use a Jupyter Notebook to practice Python concepts. On the other hand, if you're writing a program that will be used as an application or needs to be packaged, you'd typically organize it into script files or modules. It's common to use notebooks for exploration and then move code into .py files for deployment or larger projects.

Now that your environment is set up and you know how to run Python, let's go through the fundamental concepts and terminology in Python. These are the building blocks of the language that you will use in any Python program.

Essential Python Terminologies and Concepts

Below is a list of key Python terminologies every beginner should understand, along with brief explanations:

- **Interpreter:** The Python **interpreter** is the program that executes your Python code. Python is an *interpreted* language, meaning you don't compile your code into machine

language before running it – instead, the interpreter reads and runs your code line by line[36]. When you installed Python, you obtained an interpreter (accessible via the python/python3 command or IDLE). If the interpreter encounters an error in your code, it will stop and display an error message (traceback) indicating what went wrong.

- **Program:** A Python **program** is simply a script or application written in Python – a set of instructions for the computer to execute. You can think of a program like a recipe: it contains steps (commands) that the computer follows to perform a task. Even a short Python script that prints a message is a program. Applications like web browsers or games are also programs (often composed of many files and modules). In general, *a program is a sequence of instructions* that tells the computer what to do[37].
- **Syntax and Indentation:** **Syntax** refers to the set of rules that define how Python code must be written. Python's syntax is known for being clean and readable. One unique aspect is that **indentation (leading whitespace on lines)** is meaningful in Python. Instead of using braces {} or keywords like `endif` to designate code blocks, Python uses indentation levels. For example, the lines inside a function or inside an `if` statement must be indented consistently. If the indentation is wrong or inconsistent, the interpreter will raise an `IndentationError`. Indentation thus determines the structure of code blocks[38]. This requirement makes Python code visually neat and helps enforce good formatting habits. Other syntax rules include using a colon `:` to start an indented block (e.g., after `def` for a function, or after `if` conditions), and the use of line breaks to end statements (one statement per line, unless you use a backslash or other means to continue).
- **Comments:** In Python, a **comment** is text in the code meant for humans, not for execution. Comments are created by prefixing text with the `#` symbol. For example: `# This is a comment.` The Python interpreter ignores anything on a line after a `#`, so comments do not affect the program's behavior. They are used to explain code or leave notes. *Comments are code lines that will not be executed*[39]. Python also has a way to write multi-line comments or docstrings using triple quotes (`""" ... """`), commonly used to document functions or modules.
- **Variable:** A **variable** is a named storage location that holds a value. You can think of it as a container or a label for data. In Python, you create a variable by assigning a value to a name, for example: `count = 10`. Here, `count` is a variable name that refers to the value 10. Variables make it possible to store and manipulate data in programs. Internally, a variable name points to an object representing the data in memory. *Basically, a variable is a place in computer memory with an assigned name, used to store data of different types (text, numbers, etc.)*[40]. Unlike some languages, you don't declare a type for a

variable in Python – the type is inferred from whatever value you assign to it, and you can later change the value (and type) of the variable if needed.

- **Data Types (Numbers, Strings, Booleans, etc.):** A **data type** defines what kind of value a piece of data is, which determines what you can do with it. Common Python data types include:
 - **Integer (int):** whole numbers (e.g. 5, -42).
 - **Floating-point number (float):** numbers with a decimal point or in exponential form (e.g. 3.14, 2.0, 5e6 for 5×10^6).
 - **String (str):** sequence of characters used to represent text (e.g. "Hello" or 'Python'). Strings are enclosed in quotes. Python strings can be manipulated with many methods since they are a **sequence** type (you can think of them as a sequence of characters). For instance, you can find its length, slice it, or split it into parts. (Examples: "Hello"[1] gives "e", and "Hello".split("e") gives ["H", "llo"].)[41][42].
 - **Boolean (bool):** logical values True or False. Booleans often result from comparisons (like $5 < 10$ yields True) and are used in control flow (if/while conditions). In Python, True and False are reserved keywords representing these two truth values.
 - **NoneType:** a special type with a single value None, used to represent “no value” or the absence of data.

Python is *dynamically typed*, meaning you don’t need to specify the type of a variable and a variable’s type can change when you assign a new value. You can always check the type of a value by using the type() function (e.g. type(42) returns <class 'int'>).

- **Operator:** An **operator** is a symbol (or sometimes a keyword) that tells the interpreter to perform a specific operation on one or more operands (values or variables). For example, + is an arithmetic operator that adds two numbers or concatenates strings. Python supports many types of operators:
 - **Arithmetic operators** (+, -, *, /, // for floor division, % for modulus, ** for exponentiation) to perform mathematical calculations[43].
 - **Comparison (Relational) operators** (== equals, != not equals, >, <, >=, <=) to compare two values, yielding a boolean result[44].
 - **Logical (Boolean) operators** (and, or, not) to combine boolean conditions (and & or) or negate them (not)[45].
 - **Assignment operators** (like = to assign, +=, -=, etc. to modify and assign) for setting variable values[46].
 - **Identity operators** (is, is not) to check if two references point to the same object in memory.

- **Membership operators** (`in`, `not in`) to test if a value is a member of a sequence or collection (e.g. `x in my_list`)[47].

Using operators, you can form expressions like `a * b + 5` or `if (x > 0 and x < 10): ....`. The operators follow standard precedence (e.g. multiplication comes before addition, unless parentheses are used to group).

- **Conditional (If) Statement: If statements** are a fundamental control structure that let your program make decisions. An if statement executes a block of code only if a certain condition is true. In Python:

```
if condition:
    # code block
elif other_condition:
    # code block
else:
    # code block
```

You start with `if` and a condition (an expression that evaluates to `True` or `False`). If the condition is `True`, the indented block under the `if` runs; if it's `False`, the program can optionally check other conditions with `elif` (else-if) or execute a default block under `else`. For example:

```
age = 20
if age < 18:
    print("Minor")
else:
    print("Adult")
```

Here, "Adult" will be printed because the condition `age < 18` is `False`, so it goes to the `else`. Conditions often use comparison operators (`==`, `>`, `<`, etc.) and logical operators (`and`, `or`) to combine multiple criteria[48][49]. Python uses a colon `:` after the condition and indentation to mark the block of code that belongs to the `if`. Proper indentation is crucial, as mentioned earlier, to ensure the `if` body is recognized[38]. If statements allow programs to branch and perform different actions based on input or state.

- **Loop (For / While): Loops** allow you to execute a block of code repeatedly. Python has two main loop types:
- **For loop:** used to iterate over elements of a sequence (like a list, tuple, set, string, or range of numbers). In Python, a for loop looks like `for item in collection: ....`. It will assign each element of the collection to `item` and execute the indented block for each element[50]. For example:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

This will print each fruit name in the list. A for loop is ideal when you know in advance how many elements (or iterations) you need to go through.

- **While loop:** used to repeat a block as long as a condition remains true. It looks like while condition: The loop will keep running, checking the condition before each iteration, and it will stop once the condition becomes false[\[51\]](#)[\[52\]](#). For example:

```
count = 5
while count > 0:
    print(count)
    count -= 1
```

This will print 5, 4, 3, 2, 1 each on a new line, then stop when count is 0 (since the condition count > 0 is no longer true). You must ensure that the condition will eventually become false, otherwise a while loop can run forever.

Both loop types use indentation to define the loop body. You can use the keyword break to exit a loop early, or continue to skip to the next iteration. In general, *for loops* are common when iterating over known collections or ranges, and *while loops* are useful when the repetition should continue until a condition changes (which might depend on user input or some calculation). Loops are essential for tasks like iterating through data, repeating an operation multiple times, or searching through collections[\[53\]](#)[\[50\]](#).

- **Function:** A **function** is a reusable block of code that performs a specific task. You can define your own functions using the def keyword, giving the function a name and optional parameters. For example:

```
def greet(name):
    print("Hello,", name)
```

defines a function greet that takes one parameter name. Using greet("Alice") would print "Hello, Alice". Functions allow you to organize code into logical pieces and avoid repetition – you write the code once and call it whenever needed. *A function is a reusable block of code that performs a certain action (or actions). Functions are used to accomplish tasks, especially those that need to be done repeatedly, with different inputs*[\[54\]](#). Python also has many **built-in functions** (like len(), range(), etc.), and you can import functions from modules. When a function needs to produce a result, it can return a value, which then can be used by the caller. If a function doesn't explicitly return anything, it returns None by default.

- **print() function:** print() is one of Python's most basic and frequently used built-in functions. It outputs messages to the screen (or more technically, to the standard output). For example, print("Welcome to Python") will display that text. You can print variables and other data types as well. print() is very handy for showing results or for debugging by inspecting variable values. *Every Python developer learns to use print() early, as it allows us to display the result of code or messages for the user*[55]. By default, print adds a newline at the end of what it prints, so each call prints on a new line (this can be changed with an optional parameter).
- **input() function:** input() is the counterpart to print – it allows you to **receive input from the user**. When you call input(), the program will pause and wait for the user to type something and press Enter. The text the user enters is then returned as a string. You can optionally pass a string to input() to display a prompt message (e.g. name = input("Enter your name: ")). For example:

```
age_text = input("How old are you? ")
```

If the user types "21" and hits enter, then age_text will be the string "21". You might then convert this to an integer with age = int(age_text). *The input() function always returns the user input as a string (even if they typed digits)*[56]. Using input() is one way to make programs interactive and dynamic.

- **Object:** In Python (an object-oriented language), an **object** refers to an instance of a data type that encapsulates both data and functionality. Practically everything in Python is an object – integers, strings, lists, functions, even modules are objects. Each object has a type (or class) that defines what it is and what it can do. For example, the number 42 is an object of type int, and it has certain behaviors (like it can be added or subtracted). A list like [1,2,3] is an object of type list and has methods like .append(). If you define a class (blueprint) for something, the actual realized entities are objects. *In short, an object is a concrete instance of some class or data type, bundling together data and operations*[57]. For a real-world analogy, if you have a class Car (concept), then a specific car (like your dad's red Toyota) is an object (instance) of that class.
- **Class:** A **class** is like a blueprint or template for creating objects. It defines a new data type by specifying what attributes (data) and methods (functions) its objects will have. You can define a class in Python using the class keyword:

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
```

```
def drive(self, miles):
    print(f"Driving {miles} miles")
```

Here Car is a class with attributes brand and color, and a method drive. From this class, you can create objects (instances) like `my_car = Car("Toyota", "red")`. *A class can be thought of as a blueprint for objects – it defines the structure and behavior (via attributes and methods) that the objects of the class will have*[\[58\]](#). Python has many built-in classes (for example, list, dict, str are classes), and you can also create your own to model more complex data. One advantage of using classes is that you can create multiple objects from the same blueprint without repeating code, and you can leverage concepts like inheritance (one class inheriting attributes/methods from another). For beginners, it's enough to grasp that classes = blueprints, objects = actual instances.

- **Attribute:** An **attribute** is a piece of data stored in an object (or class). If we think in terms of Python code, attributes are the **variables/properties that belong to an object or class**. There are two types:
 - *Instance attribute:* a piece of data specific to an object (instance). In the Car class example, brand and color are instance attributes; each Car object has its own values for these.
 - *Class attribute:* a variable that is defined at the class level (not in any method) and is shared by all instances of the class.

Additionally, **methods** (see below) are sometimes called “behavioral attributes” or “procedural attributes” of a class. *Data attributes define what an object needs to have (state), while methods define how to interact with the object (behavior)*[\[59\]](#). For example, if you have an object person of class Person with attributes name and age, those are data attributes. If Person has a method `speak()`, that method is also considered an attribute of the class (a function attribute). In Python, you access attributes with the dot notation: e.g. `person.name` or `my_car.color`. Trying to access an attribute that doesn't exist will result in an `AttributeError`. Understanding attributes is important when dealing with objects and classes, as it helps organize data and functionality together.

- **Instance:** An **instance** is another name for an object in the context of a class. We say an object is an instance of a particular class. For example, using the Car class above, `my_car = Car("Toyota", "red")` creates an instance of Car. You can create multiple instances (objects) from one class, each with their own attribute values. *An instance of a class is an individual object that possesses the structure and behavior defined by that class*[\[60\]](#). In conversation, “instance” and “object” are often used interchangeably. The word instance emphasizes the relationship to its class (like “this is an instance of class Car”).

Checking an object's type with `type(obj)` will show you its class, and `isinstance(obj, ClassName)` returns True if the object is an instance of that class (or a subclass of it).

- **Method:** A **method** is a function that is defined inside a class and is meant to be called on objects of that class. Methods typically operate on the object's data. For example, in a list object `my_list`, `my_list.append(5)` calls the `append` method of the list, which adds a new element. Methods are described as the behavior of an object. The only difference between a method and a regular function is that a method is tied to an object (the first parameter of a method is usually `self`, referring to the instance). *Methods are similar to functions, but they belong to a particular class and are used to interact with class instances*[\[61\]](#). For instance, `str.upper()` is a method that returns an uppercase version of a string. In our Car class example, `drive()` is a method; you would call `my_car.drive(100)`, which would perhaps reduce fuel or print a message. Understanding methods is important because much of Python's functionality comes from calling methods on objects (e.g. list methods to manipulate lists, dict methods to handle dictionaries, etc.).
- **List:** A **list** is a built-in data structure in Python that holds an ordered collection of items. Lists are very flexible – they can contain elements of any type, and you can add, remove, or change items. Lists are created using square brackets, e.g. `numbers = [1, 2, 3, 4]`. The items in a list have a defined order (index starting from 0 for the first item) and the list can be modified (it's *mutable*). You can access elements by index (`numbers[0]` gives 1), slice lists (`numbers[1:3]` gives `[2,3]`), and use methods like `append()`, `remove()`, `sort()`, etc. *In summary, a list is a data type that can store a sequence of values, accessible by an index (position)*[\[62\]](#). Lists are one of the most commonly used data structures for storing collections of items.
- **Tuple:** A **tuple** is another built-in sequence type, quite similar to a list, but with one key difference: tuples are **immutable**. Once a tuple is created, its contents cannot be changed (you can't add, remove, or modify items in place). Tuples are defined using parentheses, for example `coordinates = (10.0, 20.0)` or `my_tuple = ("a", "b", "c")`. They support indexing and slicing like lists (`coordinates[0]` is 10.0). Because they are immutable, tuples are useful for representing fixed collections of items (such as a pair of coordinates, or a database record that shouldn't change). Tuples can also be used as keys in dictionaries (since immutability makes them hashable), whereas lists cannot. *In essence, a tuple is a sequence of objects, very much like a list, but defined and fixed upon creation*[\[63\]](#)[\[64\]](#). If you need an ordered collection that you won't need to modify, a tuple is a good choice.

- **Dictionary:** A **dictionary** (dict) is a built-in Python data structure that stores a collection of key–value pairs. It's also sometimes called a hash map or associative array in other languages. You create a dictionary with curly braces {}, for example:

```
student = {"name": "Alice", "age": 25}
```

Here "name" and "age" are keys, and "Alice" and 25 are their respective values. You retrieve or set values by key, e.g. `student["name"]` gives "Alice". Dictionaries are *unordered* collections (as of Python 3.7+, insertion order is preserved, but conceptually you don't rely on order) and are mutable (you can add, remove, or change entries). Keys in a dict must be unique and immutable (numbers, strings, tuples, etc. can be keys; lists cannot). *A Python dictionary stores an unordered collection of data values, where each element has a key and a value*[\[65\]](#). They are very powerful for representing structured data and performing fast lookups by key (like a word-to-definition mapping, or a username-to-user info mapping). For instance, you could have `prices = {"apple": 0.99, "banana": 0.50}` to map items to their prices.

- **Set:** A **set** is a built-in data type that represents an **unordered collection of unique elements**. Sets are like mathematical sets – they don't allow duplicates and the order of elements is not preserved. You create a set using curly braces with comma-separated values (similar to dict literal but just values, no colons), e.g. `unique_numbers = {1, 2, 5}`. You can also use the `set()` constructor to create a set from another iterable (like a list). Key features of sets:
 - Fast membership testing: you can check if an item is in a set with in very efficiently.
 - Because they only allow unique items, sets are useful for removing duplicates from a collection.
 - They support operations like union, intersection, difference (via methods or operators like `|`, `&`, `-`).

In Python, a set is an unordered collection of distinct (unique) immutable objects[\[66\]](#). For example, `{1,2,3,3}` will end up as `{1,2,3}` because duplicates are removed[\[67\]](#). Sets themselves are mutable (you can add or remove elements), but because they are unordered, you cannot access elements by index. There is also a special type called a **frozenset** which is an immutable set. Beginners commonly use sets when they need to gather unique items or perform mathematical set operations.

- **String:** We touched on strings earlier as a data type, but to elaborate: a **string** in Python is a sequence of characters used to represent text. Strings are created by enclosing characters in quotes, either single `'...'` or double `"..."` quotes. They are **immutable** (you

cannot change a string in place; operations that appear to modify a string actually create a new string). Python provides many helpful operations for strings:

- You can **concatenate** strings with + (e.g. "Hello, " + name).
- **Length:** use len(my_string) to get the number of characters.
- **Indexing and slicing:** you can access characters by index (s[0]) or substrings via slicing (s[start:end])[68]. Indices start at 0, and negative indices count from the end (s[-1] is the last character)[69].
- **Methods:** There are many built-in methods, e.g. s.lower(), s.upper(), s.split(" "), s.strip(), s.find("abc"), etc., to transform or search within the string. For example, "John Doe".split(" ") returns ["John", "Doe"][42].

Strings are fundamental for input/output, representing messages, file paths, data read from files, etc. Even though they are immutable, operations are designed to be efficient and convenient.

- **Library/Package:** In Python, a **package** (or library or module – the terms are related, see below) is a collection of pre-written code that you can include in your projects to add functionality. The Python Standard Library is the collection of packages/modules included with Python by default (for tasks like math, file I/O, data structures, etc.). Additionally, the Python community has tens of thousands of third-party packages available via the Python Package Index (PyPI). A **package** in the filesystem sense is a directory containing one or more Python modules (usually identified by an __init__.py file). However, when we say “*Python package*” colloquially, we often mean any installable library or tool. Examples include numpy (for numerical computing), pandas (data analysis), requests (HTTP requests), etc. *Python offers a rich selection of packages; any developer can create a package and publish it on PyPI for others to install*[70]. Packages are used to **re-use code** – instead of writing something from scratch, you can install a package that provides the functionality you need.
- **Module:** A **module** is a single Python file (with a .py extension) that can be imported, or a component of a larger package. When you import a module, you’re essentially loading that file and making its functions/classes available in your code. For example, Python’s standard library has a module called math (contained in math.py in the library), and you can do import math to use math.sqrt(16) etc. If you have a file utils.py with some helper functions, utils is a module you wrote. *In terms of relationship: a package may contain multiple modules (files), whereas a module is a single file. Often, you will only need to import a specific module from a package rather than the whole package at once*[71]. For instance, a large package might have submodules and you could do from package import

submodule. But if you're not creating your own packages yet, you can think of modules simply as "libraries" or files you import.

- **pip (Package Installer):** **pip** is the standard package installer for Python. It's the tool you use to **install and manage third-party libraries** from the Python Package Index (and other repositories). For example, if you want to install the requests library, you would run `pip install requests` in your command prompt. Pip will download the package and its dependencies and install them into your Python environment. *In other words, pip is the package installer for Python – you can use pip to install packages from the Python Package Index (PyPI) and other indexes*[\[72\]](#). Pip typically comes bundled with Python installers. To use it, open a terminal and type `pip install <package_name>`. You can also use it to upgrade packages (`pip install --upgrade <name>`) or uninstall (`pip uninstall <name>`). Knowing how to use pip is essential for leveraging Python's vast ecosystem of external libraries. (In environments like Anaconda, you might use `conda install` for packages available through conda, but pip can still be used there as well.)

Having reviewed these core terms and concepts, you should now have the vocabulary to understand Python code and documentation. Next, let's apply some of these concepts in practice using a Jupyter notebook.

Hands-On Practice: Using a Jupyter Notebook to Explore Python

At this point, you have Python set up and know the basic concepts. A great way to solidify your understanding is to play with code in a **Jupyter Notebook**. You can either use an online notebook (e.g. Colab) or run one locally. Here's a suggested plan for a notebook session:

1. **Basic operations:** In a new notebook, try using Python like a calculator. For example, compute `5 + 3*2`, see how operator precedence works. Try assigning variables like `x = 10`, then do `x * 2` and `x = "Hello"` to see dynamic typing.
2. **Using print and input:** Write a cell that asks for input and then prints a formatted message. (*Note:* In Colab, `input()` works, but in some notebook environments (like JupyterLab) it will prompt in the console. In any case, you can practice by just setting a variable instead of truly interactive input if needed.) For example:

```
name = input("What is your name? ")
print("Nice to meet you, " + name + "!!")
```

3. **Control flow:** Practice with if statements and loops. For instance, write a cell with an if-elif-else chain that prints different messages based on a numeric variable. Or use a for-loop to iterate over a list of names and greet each.
4. **Define functions:** Try writing a simple function, like `def square(n): return n*n`. Then call it with a few arguments (e.g. `square(5)`). Maybe write a function that given a temperature in Celsius returns it in Fahrenheit, to practice using a formula and return.
5. **Use data structures:** Create a list of some items (e.g. favorite foods), then write a loop to print each item in uppercase. Create a dictionary mapping some keys to values (e.g. fruits to colors) and retrieve a value by its key. Modify the dictionary by adding a new entry. Experiment with a set by making a list with duplicate values and converting it to a set to see duplicates removed.
6. **Explore a library:** Use the math module or another simple library to see how importing works. For example:

```
import math
print(math.sqrt(16))
```

Or use `from datetime import datetime; print(datetime.now())` to get the current time. This helps practice importing modules.

By playing with these in a notebook, you'll reinforce the terminology above. Remember, the notebook lets you run pieces of code one at a time, so you can modify a variable in one cell and use it in the next, see errors and adjust, etc., in an interactive way.

Next Steps: Loading Data and Doing Useful Work

One of the powerful things about Python is the ability to work with various types of data, including files, images, audio, and video. By now, you should be comfortable with the basics – you can write Python code, use variables, loops, and functions. As a next step, let's look at a simple example of how you might **load your own data (like an audio or video file) and perform basic tasks** with it. This will give a glimpse of Python's real-world applications.

Suppose you have a video file and you want to extract some information or frames from it. Python has libraries like **OpenCV** (Open Source Computer Vision library) that can handle video processing. For example, using OpenCV in Python:

```
import cv2
cap = cv2.VideoCapture("myvideo.mp4") # open the video file
```

```

if cap.isOpened():
    frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    print("Total frames in video:", frame_count)
    cap.release()
else:
    print("Failed to open video file")

```

In the above snippet, `cv2.VideoCapture` is used to open a video file. The OpenCV library lets you treat a video similarly to a sequence of images (frames)[\[73\]](#). We checked if the video opened successfully, then retrieved the number of frames as a basic task (you could also read frames in a loop for processing, e.g. to convert to grayscale or detect objects in each frame).

Now consider an audio file (say a .wav music file). A popular library for audio analysis is **Librosa**. Here's an example of using `librosa` to load an audio file and get its data:

```

import librosa
audio_data, sample_rate = librosa.load("audio.wav", sr=None) # load with original sampling rate
print(f"Loaded {len(audio_data)} audio samples at {sample_rate} Hz")

```

`Librosa`'s `load` function reads the audio file and returns two things: a NumPy array of audio samples and the sample rate (number of samples per second)[\[74\]](#). With this, you could, for example, compute the duration of the audio in seconds by `len(audio_data)/sample_rate`, or perform further analysis like computing a spectrogram or detecting beats. There are also simpler libraries like **pydub** or even Python's built-in `wave` module for basic audio tasks.

These examples illustrate that with a few lines of Python, you can start handling complex data types. Whether it's reading a spreadsheet, an image, or a video, there's likely a Python library available. The process usually involves: installing the library (via `pip`), importing it in your script or notebook, and then calling its functions to load or manipulate data.

Conclusion: From Basics to Building Applications

Congratulations on making it through this introduction to Python! You've learned what Python is, set up an environment, and covered fundamental programming concepts from variables and functions to loops and classes. At this stage, you should be able to write simple programs and use notebooks to experiment with code. You even saw how to bring in external libraries to work with real-world data like audio and video.

Learning programming is a bit like learning a language – at first you learn words and grammar (we covered the “words” like variables, and “grammar” like syntax/loops/functions). The next step is to practice by actually “speaking” – writing programs to solve problems or automate

tasks. Start with small projects or exercises: for example, write a program to convert units (temperature, currency), or a simple number guessing game, or analyze a small dataset.

As you become comfortable with the basics, you can explore more advanced topics or domains:

- **Data Science/Analysis:** Learn libraries such as pandas, NumPy, matplotlib, and explore how to manipulate datasets and create visualizations.
- **Web Development:** Try a web framework like Flask or Django to build a simple web app. Python's flexibility means the same fundamentals you learned apply here – you'll just be writing functions that respond to web requests instead of a loop in a notebook. (Python being used in web development is one of the reasons for its broad adoption[1].)
- **Automation/Scripting:** Use Python to automate tasks on your computer – for example, rename files in bulk, scrape data from websites (with libraries like BeautifulSoup), or send automated emails.
- **Machine Learning/AI:** Explore libraries like scikit-learn or TensorFlow once you have a handle on Python – the strong foundation in Python will allow you to grasp these libraries faster.
- **Developing and Deploying Apps:** Eventually, you might combine your Python knowledge with frameworks and cloud services to create deployable applications. Python's simplicity and the vast array of tools available mean that with time, you could write a Python program and deploy it as a cloud service or application that others can use. (For instance, many cloud services support Python natively, and you can deploy Python web apps to platforms like Heroku, AWS, etc., turning your code into a live service.)

Keep in mind that Python was created to be an easy-to-understand language, and it has a very supportive community. There are countless resources and forums (like Stack Overflow, Reddit's r/learnpython, etc.) where you can seek help and advice.

With the guide and the hands-on practice in the notebook, you're well on your way. Happy coding, and welcome to the world of Python programming! As you continue, you'll discover more *cool reasons to learn and use Python*, including the ability to quickly turn ideas into working code[18]. Good luck on your journey to eventually building that cloud-deployable app – step by step, you will get there using the skills you've started building today. Python's versatility and power will be your ally every step of the way. Enjoy!

References:

[1] [2] [3] [4] [5] [36] [37] [38] [40] [48] [49] [50] [51] [52] [53] [54] [55] [56] [62] [65] Python Terms Beginners Should Know – Part 1 | LearnPython.com

<https://learnpython.com/blog/python-terms-for-beginners/>

[6] [7] [8] [9] [10] [11] Google Collab vs Jupyter Notebook. - GeeksforGeeks

<https://www.geeksforgeeks.org/blogs/google-collab-vs-jupyter-notebook/>

[12] [16] [17] [19] [20] [21] [22] [23] [24] [25] [26] Tutorial: Installing Python on Mac – Dataquest

<https://www.dataquest.io/blog/installing-python-on-mac/>

[13] [31] [32] [33] How to install Python on Windows? - GeeksforGeeks

<https://www.geeksforgeeks.org/python/how-to-install-python-on-windows/>

[14] [15] [28] [29] [30] [34] [35] Python on Windows for beginners | Microsoft Learn

<https://learn.microsoft.com/en-us/windows/python/beginners>

[18] [41] [42] [57] [58] [59] [60] [61] [63] [64] [66] [67] [68] [69] [70] [71] Python Terms Beginners Should Know – Part 2 | LearnPython.com

<https://learnpython.com/blog/python-terms-for-beginners-2/>

[27] 4. Using Python on Windows — Python 3.13.7 documentation

<https://docs.python.org/3/using/windows.html>

[39] [43] [44] [45] [46] [47] Python Glossary

https://www.w3schools.com/python/python_ref_glossary.asp

[72] pip · PyPI

<https://pypi.org/project/pip/>

[73] Getting Started with Videos - OpenCV Documentation

https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

[74] librosa.load — librosa 0.11.0 documentation

<https://librosa.org/doc/main/generated/librosa.load.html>